Detecting and Interpreting Text Generated by Large Language Models

Murema Manganyi
University of Cape Town
Cape Town, South Africa

Abstract

Large language models such as ChatGPT and Claude can generate text that closely resembles human writing, creating a need for reliable ways to distinguish between human-written and LLM-generated text. A detector can predict whether a passage is human- or machine-generated, but it is also important to understand what in the text supports that prediction, especially when the prediction needs to be checked or justified. One way to make this process more interpretable is to use a rationale, where the prediction is based only on selected parts of the input text. We investigate whether a rationale-based detector can retain detection performance while making this selected text visible, and whether showing the same text to human readers helps them judge authorship. We train the detector on an English subset of RAID and compare it with a matched classifier that reads the full passage. The rationale model retained most of the full-context model's detection performance, and the text it selected was more useful to the predictor than random selections or selections taken from the beginning or end of the passage. However, showing the learned rationales to human readers produced no clear improvement in authorship judgements over plain text or random highlighting. The results show that evidence which is useful to a detector is not necessarily useful to a human reader, and that the two should be evaluated separately.

Keywords

machine-generated text detection, large language models, interpretability, rationales

1 Introduction

Large language models (LLMs) are artificial intelligence systems trained on large amounts of text to predict and generate sequences of words. Given some text, an LLM predicts a likely next token and repeats this process to generate a longer passage [21]. Systems such as ChatGPT, Gemini, and Claude have made this capability widely accessible, allowing users to generate fluent text across many domains with little effort. In this paper, we refer to text produced by these systems as LLM-generated text.

The task of distinguishing human-written text from text generated by a language model is known as machine-generated text (MGT) detection. This matters in settings such as education, online reviews, and misinformation, where knowing how a piece of text was produced may affect how it is used or trusted [28]. Because LLM-generated text can closely resemble human writing, making this distinction is not always easy.

Alongside prediction accuracy, we study how understandable a detector's decision is. If a detector predicts that a passage is machine-generated, the prediction alone does not show what in the passage supported that result. This matters when someone needs to check or justify the prediction. Our approach is to make the text used for the prediction visible.

We use rationales to do this. A rationale is a selected part of the input used to make a prediction. In our classifier, a selector reads the passage and chooses token positions, while a separate predictor receives token content only from the selected positions [2, 13]. The selected text can then be shown alongside the prediction. Because the predictor does not receive the full passage, we also need to measure how much detection performance is lost when it is restricted to the rationale.

This still does not tell us whether the selected text is useful to a reader. A rationale may contain enough information for the predictor to classify a passage without making it clear to a person why those words suggest human or machine authorship. GLTR showed that token-level information can help people identify machine-generated text [9]. We also test whether the rationales learned by our detector help people judge authorship.

To test these questions, we use the RAID dataset [8]. We restrict it to English passages from four domains and six generators, use sampled generations without adversarial modification, and split the data so that text from the same source does not appear across the training, validation, and test sets. We train a rationale-based BiLSTM and compare it with a matched BiLSTM that reads the full passage. We test whether the learned selections are more useful to the predictor than random selections or selections taken from the beginning or end of the passage. We also test how the prediction changes when the selected text is kept or removed, and how similar the selected text is across independently trained models. A second pair of models uses Transformer encoders to see whether the same pattern appears with a different architecture. Finally, we conduct a human study in which participants judge authorship from plain text, learned rationale highlighting, or random highlighting.

The study addresses the following research question:

Can a rationale-based classifier retain detection performance relative to a matched full-context classifier while exposing evidence that matters to its prediction, and does showing that evidence help human evaluators judge authorship?

We answer this question through three objectives:

(1) Compare the detection performance of a rationale-based classifier with a matched full-context classifier.
(2) Evaluate whether the selected tokens are useful to the predictor, whether the prediction depends on them, and whether independently trained models select similar evidence.
(3) Evaluate whether showing the selected tokens to human readers improves their authorship judgements compared with plain text and random highlighting.

The rationale model retained most of the detection performance of the full-context model, and the learned selections were more useful to the predictor than random selections or selections taken from the beginning or end of the passage. Showing the learned rationales did not clearly improve authorship judgements over plain text or random highlighting. The selected text was therefore useful to the detector, but we found no clear evidence that showing the same text helped people make the same judgement. The main contribution of this study is that we evaluate both questions together instead of assuming that a rationale that works for the detector will also be useful to a human reader.

2 Related Work

This section first reviews approaches to machine-generated text detection and the limitations that motivate interpretable detection. We then discuss explainable text classification and rationale-based models, before considering how researchers evaluate rationales for faithfulness, stability, and usefulness to human readers.

2.1 Machine-Generated Text Detection

Machine-generated text detection aims to determine whether a passage was written by a person or generated by a language model. There are several ways to do this. One option is to train a classifier on labelled examples of human and machine-generated text. Neural detectors often do this by fine-tuning a pre-trained language model such as RoBERTa so that it learns patterns that help separate the two classes [14]. These patterns may include differences in wording, style, or how likely certain sequences are to occur.

Other methods do not train a separate classifier on labelled examples. Instead, they look for statistical properties of generated text. DetectGPT, for example, checks how the probability of a passage changes when the text is slightly rewritten [18]. The idea is that text produced by a language model often behaves differently under these small changes than human text. Watermarking takes a different approach. Rather than analysing the text after it has been generated, it adds a known statistical pattern during generation so that the text can later be recognised [12]. This only works when the generator used the watermark in the first place.

The dataset used to test a detector matters just as much as the detection method. A detector may perform well on text from one generator or one domain and perform very differently when those conditions change. MULTITuDE is a multilingual collection of human and machine-generated text designed to test how detectors transfer across languages and generators [16]. DetectRL collects human and machine-generated passages from several domains and also tests detectors after the text has been modified [29]. RAID contains passages from several domains and generators and includes different decoding strategies and attacks [8]. We use a filtered English subset of RAID in this study because it gives us human and machine-generated text from several different sources under one common dataset.

The bulk of research on MGT detection has focused on prediction accuracy and on how detectors perform under different conditions [28]. These evaluations tell us whether a detector works, but they say little about what in a particular passage supports its prediction. If a prediction needs to be checked or justified, that information also matters. This is the gap that motivates interpretable approaches to detection.

2.2 Explainable AI for Text Classification

Explainable AI studies how information about a model's prediction can be made easier to understand or inspect. One useful distinction is when the explanation is produced. In ante-hoc methods, interpretability is built into the predictive model itself. In post-hoc methods, the prediction is made first, and a separate process is used to explain it afterwards [4]. A linear model whose feature weights can be read directly is a simple example of the ante-hoc case. A model that is explained after it has already made its prediction is post-hoc.

Two common post-hoc methods are LIME and SHAP. LIME stands for Local Interpretable Model-agnostic Explanations. Given one prediction, it makes small changes to the input many times. For text, this can involve removing words and observing how the prediction changes. It then fits a simple model to the classifier's behaviour around that example, with the model's weights showing which parts of the input had the strongest local effect on the prediction [22]. SHAP stands for SHapley Additive exPlanations. It comes from a method in game theory for dividing a reward among players according to their contribution. SHAP applies the same idea to a model's input features and estimates how much each feature contributes to a particular prediction [15]. Neither method changes the classifier being explained. Both are applied after the classifier has made its prediction.

This is also an important limitation of post-hoc explanation. The classifier has already processed the input before the explanation is produced, so the explanation does not control what information was available when the prediction was made. Rudin [23] argues that post-hoc explanations can misrepresent how a black-box model reached a decision, particularly when the decision needs to be justified. The next section considers models where the information shown as the explanation is part of the prediction process itself.

2.3 Rationale-Based Interpretability

A rationale is a selected part of the input that is used to make a prediction. In the setup introduced by Lei et al. [13], one model acts as the selector and chooses which parts of the text to keep. A second model, the predictor, makes its decision from the selected text. The two are trained together, which allows the selector to learn what information is useful to the predictor. The rationale is therefore part of the prediction process rather than an explanation added afterwards.

The selector has to decide whether each part of the input should be kept or discarded. This creates a training problem because gradients do not pass through sampled binary choices, so the selector cannot be trained by ordinary backpropagation. Lei et al. address this using REINFORCE [27], which estimates how the selector should change from the predictions made under its sampled selections. Bastings et al. [2] later replace the Bernoulli selector with Hard Kumaraswamy, or HardKuma, gates. These gates allow gradient-based training while still being able to take exact zero and one values. A zero gate removes that token from the predictor, while a one

gate keeps it. Bastings et al. also control how much of the input is selected by training towards a chosen selection rate.

Controlling the amount of selected text matters because a selector with no restriction could simply keep the whole input. Paranjape et al. [19] address the same problem using an information bottleneck. Their model learns a sparse mask while balancing how much of the input is retained against how well the selected text predicts the label. This gives another way of controlling the length of the rationale rather than leaving the selector free to keep as much text as it wants.

Jointly training the selector and predictor creates another problem. The two models can adapt to each other during training. If the predictor learns to use the evidence that the selector currently gives it, this rewards the selector for continuing to choose similar evidence. Yu et al. [30] call this interlocking. A model can therefore achieve good prediction performance without showing that its selected rationale is the only, or even the most useful, way of supporting that prediction.

Rationale models are ante-hoc, as described earlier, because the selection is made before the final prediction. The predictor only receives the selected token content, which gives the highlighted text a direct role in the decision. Jain et al. use the same general principle in models where the final classifier operates only on the extracted text [11]. Attention weights do not impose the same restriction. Serrano and Smith [24] show that tokens receiving high attention are not always the tokens whose removal changes the prediction. Restricting the predictor to a rationale therefore gives us a stronger link between the text shown and the information used for prediction. It does not, however, tell us whether the selected rationale is sufficient, stable, or useful to a human reader. These questions require their own evaluation.

2.4 Rationale Evaluation

A model that selects text can always produce something to show a reader. Whether that selection means anything is a separate question, and the literature splits it into several distinct properties.

Faithfulness asks whether the predictor actually depends on the selected text. DeYoung et al. [6] test this with sufficiency and comprehensiveness. Sufficiency keeps only the rationale and measures how much the predictor's confidence changes. A smaller drop means the rationale retains more of the original prediction. Comprehensiveness removes the rationale instead and measures how much confidence falls. A larger drop means the selected text carried more of what the prediction depended on. Chan et al. [5] show that these interventions feed the model inputs unlike anything it saw during training, which can distort the resulting scores, so the values need interpreting with that limitation in mind.

Informativeness asks a narrower question. A rationale can pass the tests above simply because it is long enough, regardless of which words it contains. The usual way to check this is to compare the learned selection against alternative selections of the same size, such as randomly chosen tokens or the first tokens of the passage. If the learned selection performs better under this comparison, the improvement comes from which words were kept rather than how many.

Stability asks whether the selection is reproducible. Two models trained on the same data can reach the same prediction from different evidence, and a rationale that depends heavily on the random seed is harder to treat as the reason for a decision. Overlap between the token positions selected by independently trained models is one way to measure this directly.

The final question is whether the selected text helps the person reading it. GLTR displays token-level information from a language model, using colour to show how expected each token is [9]. Gehrmann et al. found that this display increased human detection accuracy from 54.2% to 72.3%. GLTR gives the reader information about why a token may look machine-generated, while a learned rationale shows which text the detector selected. Whether that difference matters for a reader is untested.

3 Methodology

This section first describes the dataset and models used to compare full-context and rationale-based detection. We then explain how context length, rationale budget, and model settings were fixed on validation data, before describing the human study and the measures used to evaluate detection performance, rationale quality, and human usefulness.

3.1 Dataset

Several datasets already exist for training and evaluating machine-generated text detectors across different domains and language models [16]. We chose to use an existing dataset for this study. Generating the machine-written text ourselves would require decisions about which models, prompts and decoding settings to use, and each of those decisions would shape the resulting data.

We use RAID [8], a dataset containing human and machine-generated passages across several domains, generators, decoding strategies and attack settings. From RAID, we extract an English subset containing 8,160 passages, split evenly between human and machine-written text. The subset covers four domains, abstracts, books, news and poetry, with 2,040 passages from each. The machine-written half contains six generator labels, `gpt2`, `mistral`, `mistral-chat`, `llama-chat`, `chatgpt` and `gpt4`, with the same number of examples from each generator. The four domains differ in genre and style, and the generators include both base and chat-tuned models. We keep `chatgpt` as the label supplied by RAID and leave the underlying model version unspecified.

The study focuses on English text, so we keep only English passages. For the machine-written examples, we keep generations produced using sampling so that the decoding strategy remains fixed across the dataset. RAID also contains adversarially modified passages, where generated text has been changed to make detection more difficult. We filter these out because adversarial robustness addresses a different problem from the one studied here. Passages shorter than 80 whitespace-delimited words are also removed so that very short passages do not leave the models with only a small amount of text to work with. The 80-word threshold was fixed during dataset construction, before model validation.

We split the data by source because several human and machine-generated passages can come from the same underlying material. Keeping all passages from the same source in one partition prevents

Table 1: Composition of the final English RAID subset.

Split	Human	Machine	Total
Train	2,880	2,880	5,760
Validation	600	600	1,200
Test	600	600	1,200
Total	4,080	4,080	8,160

related content from appearing across the training, validation and test sets. This keeps the test set separate from source material already represented during training.

We treat each (domain, source_id) pair as one group and assign the complete group to the training, validation or test split. The final audit confirms that source IDs, row IDs and exact texts are disjoint across the three partitions. Every domain and generator appears in all three splits, so the final test set measures performance on new passages from the domains and generators represented during training. Generalisation to unseen domains or generators falls outside the scope of this experiment.

3.2 Models

The main model in this study uses a BiLSTM to select part of a passage before a second BiLSTM makes the prediction from the selected text. We refer to this as the rationale model. We compare it with a full-context BiLSTM built from the same predictor architecture. By full-context, we mean that the model receives every visible token in the passage. Comparing the two lets us measure how much detection performance changes when the predictor is restricted to the selected text.

We also use a TF–IDF logistic regression baseline to measure how well a simple lexical detector performs on the same data. Finally, we repeat the full-context and rationale comparison using Transformer encoders. This lets us see whether the same pattern appears with a different neural architecture based on self-attention.

3.2.1 Lexical baseline. Solaiman et al. [25] use TF–IDF word unigrams and bigrams with logistic regression to distinguish human-written WebText from text generated by GPT-2. Our task asks the same kind of human-versus-machine classification question, so we use this as a simple lexical baseline against which to compare the neural models.

Our baseline uses word unigrams and bigrams together with character sequences of three to five characters kept within word boundaries. The word features capture word choice and short phrases. The character representation captures patterns such as punctuation and word endings that the word representation can miss. We compare the word, character, and combined representations on validation data before fixing the final baseline. The exact vocabulary cut-offs and logistic-regression settings are reported in the Supplementary Information.

3.2.2 Neural representation. To make the neural-model comparisons fair, all four models use the same tokenisation and word representations. We separate words from punctuation while preserving the original case. We also store the character positions of the tokens in the original passage. These positions allow a token selected by the rationale model to be highlighted in the original text shown during the human study.

The vocabulary is the set of token forms that the model represents directly. We build it from the training split, so the input representation is fixed before validation and testing. Each token is then represented by a 300-dimensional GloVe vector [20]. GloVe provides pre-trained word representations learned from patterns of word co-occurrence. The 300-dimensional setting means that each word starts as a vector of 300 learned values. Bastings et al. [2] also use 300-dimensional GloVe representations with their BiLSTM rationale model. In our models, these vectors continue to update during training so that they can adapt to the detection task.

When a token has no matching GloVe vector, we try its lowercase form for the embedding lookup while keeping the original token in the passage.

Each neural model reads at most the first L tokens of a passage. We refer to these as the visible tokens. A longer passage is therefore classified from its first L tokens. Section 3.3 explains how L is chosen using validation data.

3.2.3 Full-context BiLSTM. The full-context BiLSTM provides the reference for the main rationale comparison. It receives every visible token in the passage. The token embeddings pass through a bidirectional LSTM with 128 hidden units in each direction. We average the resulting states across the valid token positions and apply dropout of 0.2 before a linear layer predicts whether the passage is human- or machine-generated.

The full-context BiLSTM is the same model as the predictor inside the rationale model. The rationale model then adds a selector in front of that predictor, and the selector has its own parameters. This means the two complete systems do not have the same number of parameters. The comparison is therefore between the predictor on its own, where it can use the full passage, and the same predictor placed after a selector, where it can only use the text that the selector keeps.

3.2.4 Rationale BiLSTM. The rationale model follows the selector–predictor structure introduced by Lei et al. [13] and developed with differentiable binary variables by Bastings et al. [2]. The selector reads every visible token and decides how strongly each token should be passed to the predictor. We call the value controlling this decision a *gate*. A gate of zero blocks the token embedding, while a gate of one passes the complete embedding to the predictor.

The selector and predictor are separate BiLSTMs with their own weights, while sharing the same embedding table. If e_i is the embedding for token i, the selector first produces a contextual representation for that position,

$$h_i^{(s)} = \mathrm{BiLSTM}_s(e_{1:n})_i.$$

A small learned mapping then turns $h_i^{(s)}$ into two positive values, a_i and b_i. These values control the shape of the distribution used to choose the gate,

$$(a_i, b_i) = g_\theta(h_i^{(s)}).$$

The selector needs to learn which tokens to keep during training. A direct zero-or-one choice interrupts ordinary backpropagation, because a small change to the selector parameters gives no smooth

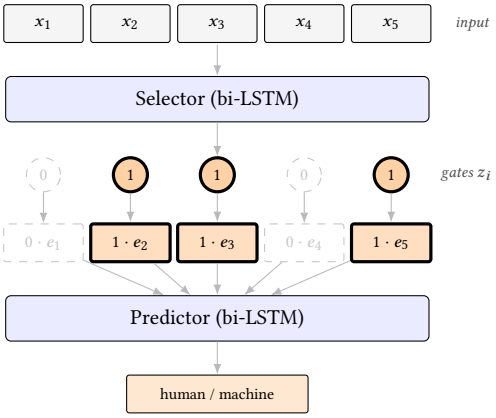


Figure 1: The selector–predictor model at inference. The selector reads every visible token and opens a gate on the ones it keeps, shown with a solid heavy border. Dropped positions, shown dashed, are zeroed rather than removed, so the predictor still sees how long the passage is and where the gaps fall but gets no token content from them. During training the gates are stochastic HardKuma samples.

change in that binary decision. HardKuma gives us a way around this. During training we sample

$$z_i \sim \text{HardKuma}(a_i, b_i),$$

where z_i is the gate for token i. The HardKuma construction starts with a Kumaraswamy sample, stretches it beyond the interval $[0, 1]$, and clips the result back to that interval [2]. This gives the training procedure a gradient path through the sampled value while still allowing gates to reach exact zero and one.

The gate is then applied directly to the token embedding,

$$\tilde{e}_i = z_i e_i.$$

A gate of zero therefore turns that position into a zero vector. A gate of one passes the original embedding. The predictor BiLSTM reads the resulting sequence,

$$h^{(p)}_{1:n} = \text{BiLSTM}_p(\tilde{e}_{1:n}),$$

and the predictor states are pooled and passed to the final human-versus-machine classifier.

Dropped positions stay in their original locations as zero vectors. The predictor therefore retains the passage length and the positions of the gaps, while the token identity and embedding content at those positions are removed. The selector itself reads the complete visible passage before choosing the gates. Figure 1 shows this information flow.

A selector trained only to improve classification has an easy way to preserve information: it can keep more of the passage. We therefore give the selector a target rationale budget, which specifies the proportion of visible tokens that should remain available to the predictor. This keeps the rationale sparse and also lets models be compared at a known amount of selected text. Bastings et al. [2] similarly train towards a prescribed selection rate using Lagrangian relaxation.

Our training objective combines the classification loss with this selection-rate constraint. Let $r(z)$ be the expected proportion of selected tokens and let r^* be the target proportion. Their difference is

$$v = r(z) - r^*.$$

The complete loss is

$$\mathcal{L} = \mathcal{L}_{\text{cls}} + \lambda v + \frac{\mu}{2} v^2.$$

The classification term, $\mathcal{L}_{\text{cls}}$, trains the model to predict the correct authorship label. The value v tells the controller how far the current selection rate is from the target. During training, λ is updated using this rate difference, while the squared term makes larger departures from the target increasingly costly. Together, these terms train the selector to find text that supports the prediction while keeping the amount of selected text close to the chosen budget.

The target-rate idea follows Bastings et al. [2]. The quadratic term, controller warm-up, smoothed rate estimate and bounded multiplier are adaptations used in our final implementation. Their exact settings are reported in the Supplementary Information.

During validation and testing, selection is deterministic. For a passage with n visible tokens and a target proportion r^*, we keep

$$k = \lceil r^* n \rceil$$

tokens. The selector scores each visible position and the k highest-scoring positions form the rationale. This gives every passage the requested selection budget at evaluation.

3.2.5 Transformer comparison. We replaced the BiLSTM encoders with Transformers to see whether this would affect detection performance. Transformers use self-attention to combine information across positions in a sequence and have achieved strong performance on sequence-modelling tasks such as machine translation [26]. Our Transformer models are also larger than the corresponding BiLSTM models, giving us a larger alternative for the same detection task. We first train a full-context Transformer and then use Transformer encoders in the rationale model to see how the model performs when prediction is restricted to selected text.

As with the BiLSTM models, both Transformer models are trained from scratch. The full-context Transformer uses 300-dimensional token embeddings and learned positional embeddings. These pass through two pre-normalised Transformer encoder layers with four attention heads and a 600-dimensional feed-forward layer. The final representations are averaged across the valid token positions and passed to the classifier. We use dropout of 0.2.

For the rationale model, the BiLSTM selector and predictor are replaced with separate Transformer encoders that share the token embedding table. The selector chooses which token positions are passed to the predictor. Dropped positions are also blocked from acting as attention keys and values, which stops the predictor from recovering their token content through self-attention.

The full-context Transformer has 4.6 million trainable parameters, compared with 3.4 million for the full-context BiLSTM. The rationale Transformer has 6.2 million parameters, compared with 3.9 million for the rationale BiLSTM. The rationale Transformer is larger because it contains separate selector and predictor encoders,

while the embedding table is shared between them. The full-context Transformer therefore lets us see whether the larger self-attention model improves detection performance. The rationale Transformer works in the same way as the rationale BiLSTM, with a selector choosing the text passed to the predictor. This lets us see how the Transformer performs when its prediction is based on the selected text.

3.3 Experimental Design

Our study asks whether a detector can make its prediction using only the parts of a passage that are most important for the decision, while still maintaining good detection performance and making that evidence visible. To test this, we vary how much of each passage is available to the model and how much of that text the rationale model is allowed to keep. We then compare the rationale model with a model that can use the full visible passage, and compare the learned rationales with other selections of the same size.

3.3.1 Context length and rationale budget. The context length determines how much of each passage is available to the neural models. Each passage can be longer than the number of tokens the model can process, so we keep the first L tokens, where L is the context length. A context length of 256 therefore gives the model at most the first 256 tokens of a passage. Increasing the context length gives the model more of the passage to work with, while also increasing the time and memory needed for training.

We test context lengths of 256, 512 and 1,024 tokens using the full-context BiLSTM. We use the full-context model here so that the amount of text available to all neural models is fixed before we introduce the selector. We compare validation machine-class F1, the proportion of passages that fit completely within each context length, and training time. At 512 tokens, 91% of the validation passages fit in full and the validation F1 is close to the result at 1,024 tokens, while training takes roughly half as long. We therefore use a context length of 512 tokens for the final neural models.

Once the context length is fixed, the rationale budget determines how much of the visible text the selector is allowed to pass to the predictor. For example, with a context length of 512 and a budget of 40%, the predictor can receive about 205 tokens from a passage that reaches the context limit. The selector decides which of those tokens are kept.

The budget affects both how selective the rationale has to be and how much information reaches the predictor. With a smaller budget, the selector has to choose fewer tokens. A larger budget gives the predictor more of the passage. Previous rationale models also control the proportion of text selected during training [2]. We test budgets of 20%, 30% and 40% with the context length fixed at 512 tokens. The 40% budget gives the highest validation machine-class F1 and is used for the final rationale model.

3.3.2 Rationale controls. A rationale model can perform well simply because the predictor is given enough text. We therefore need to check whether the particular tokens selected by the model matter. We compare each learned rationale with three other selections containing exactly the same number of tokens.

The random control selects positions from across the passage. The first-token control keeps tokens from the beginning of the passage, and the last-token control keeps tokens from the end. For the random control, we generate five fixed random masks and average the predictor probabilities across them.

Each control gives the predictor the same number of tokens as the learned rationale. We can therefore compare the information in the positions chosen by the selector with the information available from other positions in the same passage.

Table 2: Training settings used for the neural models.

Setting	BiLSTM	Transformer
Seeds	42, 43, 44	42, 43, 44
Maximum epochs	25	25
Batch size	32	32
Optimizer	Adam	Adam
Learning rate	4×10^{-4}	4×10^{-4}
Dropout	0.2	0.2
Gradient clipping	5	5
LR warm-up	None	10%

3.3.3 Training and model selection. We train each model several times because training can produce slightly different results from different random initialisations. The full-context BiLSTM, rationale BiLSTM, full-context Transformer and rationale Transformer are trained with seeds 42, 43 and 44. Their main training settings are shown in Table 2.

During training, we save versions of the model at different points. We refer to each saved version as a checkpoint. We select the checkpoint with the highest machine-class F1 on the validation set, so the test set is kept separate from model selection.

For the rationale model, checkpoint selection also considers how much text is selected. A checkpoint is eligible only when its deterministic validation selection rate is at most $1.25r^*$, where r^* is the target rationale budget. This prevents us from selecting a model that performs well by keeping considerably more text than the experiment allows.

For the Transformer models, we test learning rates of $\{10^{-4}, 2 \times 10^{-4}, 4 \times 10^{-4}\}$ using seed 42 and choose the one with the highest validation F1. We then keep that learning rate fixed for seeds 43 and 44 so that the three runs use the same configuration.

For the TF–IDF baseline, we compare the word, character and combined representations on validation F1. The combined representation performs best and is used for the final baseline.

To validate the final implementation, we check that the rationale model is using only the text selected by the selector and that the rest of the experimental pipeline behaves as intended. We verify that padding does not change predictions and that the deterministic selector keeps the expected number of tokens. We also change the content of excluded tokens and confirm that this does not affect the predictor's output. Finally, we check that the selected token positions map back to the correct text, which is important because these positions are later used to produce the highlights shown in the human study.

3.4 Human Study Design

To answer the question of whether the rationales are interpretable by human evaluators, we conduct a human study to see whether showing them helps people distinguish human-written from machine-generated text. Twenty-five participants take part in the study. Each participant sees ten passages and decides whether each one is human-written, machine-generated, or whether they cannot tell.

3.4.1 Passages and highlighting. The study uses 20 passages from the test set, with ten human-written and ten machine-generated passages. We include passages that the detector gets right and passages that it gets wrong. Five are human-written passages classified correctly as human, five are machine-generated passages classified correctly as machine, five are human-written passages classified as machine, and five are machine-generated passages classified as human. This gives us examples of the rationales behind both correct and incorrect predictions.

The highlights come from the final rationale BiLSTM with a context length of 512 tokens and a rationale budget of 40%. We use the seed-44 model because its validation F1 is closest to the mean across the three training seeds, making it the most representative run for the human study. For each passage, the highlighted text is the text selected by the model and passed to the predictor.

A passage can be shown in three ways. It can appear as plain text, with the rationale selected by the model highlighted, or with the same number of tokens highlighted at random. We refer to these as the Plain, Detector and Random conditions. The Random condition lets us check whether any benefit comes from the particular text selected by the detector, or simply from having parts of the passage highlighted.

3.4.2 Study procedure. We create six study forms, with ten passages on each form. Across the six forms, each of the 20 passages appears once as plain text, once with the detector's rationale highlighted, and once with random highlighting. Each participant completes one form and sees any particular passage only once. This allows different participants to judge the same passage in each of the three forms without one person seeing the same passage more than once.

Passages from the same source are also kept apart on a form, so participants do not repeatedly see closely related text.

After deciding whether a passage is human-written or machine-generated, participants rate how confident they are in their answer on a scale from 1 to 5. When a passage contains highlighting, they also rate how helpful and how relevant they found the highlighted text, again on a scale from 1 to 5.

Each form begins with a practice item so that participants can see how the task works before starting the study. The forms also contain an attention check.

3.5 Evaluation

We evaluate two aspects of the models. First the detection performance, whether they can differentiate machine generated from human written. We also evaluate the evidence, the rationales they produce, how useful they are to the predictor and how helpful they are to human evaluators.

3.5.1 Model Performance Evaluation. We use F1 as our main measure of classification performance. F1 combines precision and recall using their harmonic mean, so a high F1 requires the detector to perform well on both. Precision measures how often a passage predicted as machine-generated actually is machine-generated, while recall measures how much of the machine-generated text the detector identifies.

These measures are based on true positives, false positives, false negatives and true negatives, with machine-generated text treated as the positive class. A true positive is a machine-generated passage correctly classified as machine, while a false positive is a human-written passage incorrectly classified as machine. A false negative is a machine-generated passage classified as human, and a true negative is a human-written passage correctly classified as human.

We also report the Area Under the Receiver Operating Characteristic curve (AUROC). AUROC measures how well the model separates machine-generated from human-written passages using its prediction scores. The Receiver Operating Characteristic curve compares the true-positive rate with the false-positive rate as the classification threshold changes. AUROC summarises this curve into a single score, with higher values showing better separation between the two classes.

The full-context BiLSTM, rationale BiLSTM, full-context Transformer and rationale Transformer are each trained with seeds 42, 43 and 44. We report the mean and sample standard deviation across the three runs to show how consistent their performance is across training runs.

Our main comparison is between the rationale BiLSTM and the matched full-context BiLSTM. Statistical comparisons between NLP systems should account for uncertainty in the reported difference [7]. We therefore use 10,000 paired bootstrap resamples of the test set and report a 95% confidence interval for the difference in F1. The resampling is paired because both models are evaluated on the same passages.

Previous work has shown that machine-generated text detection can vary across text domains and generators [8, 18]. We therefore also evaluate the models across the four domains and six generators in our test set. We report F1 for domains because each domain contains both human and machine passages. Generator groups contain only machine-generated passages, so we report recall, which measures how much of each generator's text the detector identifies.

3.5.2 Interpretability Evaluation. For the rationales, we want to know whether the selected text is useful to the predictor and whether it is useful to a human reader. We first compare the learned rationales with random selections and with selections from the beginning and end of each passage. Each comparison keeps the same number of tokens as the learned rationale. If the model performs better with the learned rationale, the particular text selected by the rationale model carries more useful evidence than these alternative selections.

We also measure how much the prediction depends on the selected text using sufficiency and comprehensiveness, following DeYoung et al. [6]. Sufficiency asks whether the selected text is enough for the predictor to make its decision. Comprehensiveness asks how much the prediction changes when that text is removed. For the class c predicted from the learned rationale, we calculate

$$\text{Sufficiency} = p(c \mid \text{all}) - p(c \mid \text{rationale}),$$

$$\text{Comprehensiveness} = p(c \mid \text{all}) - p(c \mid \text{complement}).$$

A sufficiency score close to zero means that the rationale gives the predictor similar confidence to the full visible passage. A higher comprehensiveness score means that removing the rationale causes a larger drop in confidence. The predictor is trained on selected text, while the all-visible and complement interventions give it input patterns that differ from those seen during training [5]. We therefore interpret these measures together with the matched rationale controls.

We check whether the selected rationales are stable across training runs. Different random seeds can lead models to make similar predictions while selecting different evidence [17]. We compare the token positions selected by seeds 42, 43 and 44 using Jaccard similarity,

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}.$$

A score of 1 means that the same token positions were selected, while a score of 0 means that none were shared. We also report how often the three runs give the same predicted class. This lets us compare how consistent the selected evidence is with how consistent the final predictions are.

For the human evaluation, we measure whether participants correctly identify each passage as human-written or machine-generated. A *Cannot tell* response is counted as incorrect for this measure and is also reported separately. We also report confidence and, for highlighted passages, the helpfulness and relevance ratings.

Because each participant judges several passages and each passage is judged by several participants, we analyse accuracy using logistic regression with condition and item position as predictors, with standard errors clustered by participant and passage [3]. Item position is included because the three conditions are not distributed evenly across positions on the six study forms.

We make two planned comparisons, Detector against Plain and Detector against Random. Running more than one statistical test increases the chance of finding a difference by chance, so we apply Holm correction across the two comparisons [10]. Holm correction adjusts the significance required for the tests while controlling the probability of a false positive across the two comparisons.

4 Results

4.1 Detection Performance

Table 3 shows performance on the final test set. The rationale BiLSTM retains most of the performance of the matched full-context BiLSTM, reaching 94.5% F1 compared with 95.5%.

Across 10,000 paired bootstrap resamples, the rationale-minus-full BiLSTM F1 difference was −0.0097, with a 95% confidence interval of [−0.0172, −0.0023]. We use an F1 difference of 0.02 as a reference margin [1]. Using this margin as a reference, the confidence interval remains within a two-point F1 decrease, suggesting that the performance loss is small relative to this benchmark.

Table 3: Detection performance on the final RAID test set. Neural results are mean ± SD across three seeds.

Model	Acc.	Prec.	Rec.	F1	AUROC
TF–IDF	94.9%	95.9%	93.8%	94.9%	98.9%
Full BiLSTM	95.5% ± 0.4%	95.2% ± 1.0%	95.8% ± 0.3%	95.5% ± 0.4%	98.9% ± 0.1%
Rationale BiLSTM	94.5% ± 0.4%	94.1% ± 0.5%	95.0% ± 1.3%	94.5% ± 0.4%	98.6% ± 0.0%
Full Trans.	96.3% ± 0.2%	95.8% ± 0.8%	96.9% ± 0.7%	96.4% ± 0.2%	98.9% ± 0.0%
Rationale Trans.	93.2% ± 0.4%	92.7% ± 1.2%	93.8% ± 0.9%	93.2% ± 0.3%	97.7% ± 0.3%

The Transformer results show a different pattern. The full-context Transformer had the highest F1 at 96.4%, while the rationale Transformer reached 93.2%, below the rationale BiLSTM at 94.5%.

Performance also varied across domains and generators. The rationale BiLSTM reached above 93% mean F1 in all four domains, with poetry lowest at 93.5%. Across generators, GPT-4 had the lowest machine recall at 92.0%.

4.2 Rationale Evidence

Table 4 compares the learned rationales with the equal-count controls. The rationale BiLSTM reached 94.5% F1 using the learned rationales. This dropped to 86.6% with random selections, 77.5% when using the first tokens and 78.6% when using the last tokens. Each condition keeps the same number of tokens for a given passage, so the difference comes from which text is selected.

Table 4: Rationale diagnostics for the final BiLSTM. F1 is reported as mean percentage ± seed SD. Sufficiency and comprehensiveness are mean ± seed SD. Stability measures compare exact visible-token positions across the three seeds.

Measure	Result
Learned-rationale F1	94.5% ± 0.4%
Random-mask F1	86.6% ± 4.4%
First-token F1	77.5% ± 2.0%
Last-token F1	78.6% ± 5.2%
Sufficiency	−0.2270 ± 0.1053
Comprehensiveness	0.5164 ± 0.0528
Pairwise Jaccard	0.6329
Prediction agreement	94.1%

Removing the selected rationale caused predicted-class confidence to drop by 0.5164 on average. When the predictor saw only the rationale, its confidence was higher than when it saw the full visible passage, giving a mean sufficiency score of −0.2270. The predictor was trained on selected text, while the full visible passage differs from the inputs it normally sees. We therefore do not take the negative sufficiency score to mean that the rationale is better than the full passage.

The selected token positions were similar, but not identical, across the three training runs. Their mean pairwise Jaccard similarity was 0.6329. The three runs gave the same predicted class for 94.1% of the test passages. The predictions were therefore more consistent across runs than the exact evidence selected for them.

4.3 Error Analysis

We inspect a false positive and a false negative from the final rationale model. Figure 2 shows excerpts from the selected rationales. The full examples are given in the Supplementary Information.

False positive Human → Machine, $p(M) = 0.720$

Here a method is presented by which the fact that multiple scattering encodes structure factor phases in the diffracted intensities is being used for solving the crystallographic phase problem. The retrieval of both amplitudes and phases of electron structure factors from diffraction patterns recorded with varying angle of incidence will be demonstrated.

False negative Machine → Human, $p(M) = 0.0004$

In this paper, we study tangentially biharmonic Lagrangian H-umbilical submanifolds in complex space forms. We provide a necessary and sufficient condition for a submanifold to be tangentially biharmonic Lagrangian H-umbilical. Moreover, we obtain various characterizations and properties of these submanifolds.

Figure 2: Excerpts from two errors made by the final rationale model. Highlighted text was selected by the rationale model.

Both examples are scientific abstracts. In the false positive, the model selects terms around methods, diffraction and structure factors and predicts machine authorship. In the false negative, it also selects academic framing such as "In this paper", "necessary", "sufficient condition" and technical terminology, but predicts human authorship. The two examples show that similar kinds of academic language can appear in rationales supporting different predictions.

4.4 Human Evaluation

The human study produced 250 judgements from 25 participants. Table 5 shows the accuracy under each condition, together with the adjusted accuracy estimates and the proportion of responses where participants selected *Cannot tell*. Overall, participants performed similarly across the three conditions. Detector highlighting did not produce higher accuracy than either Plain text or Random highlighting.

Table 5: Human-study results by condition.

Condition (N)	Raw acc.	Adjusted acc. (95% CI)	Cannot tell
Plain (83)	47.0%	48.9% [21.6, 76.2]	9.6%
Detector (82)	45.1%	42.5% [21.9, 63.2]	8.5%
Random (85)	44.7%	45.4% [28.5, 62.2]	15.3%

We analysed accuracy using logistic regression with condition and item position as predictors, using two-way clustered standard errors over participants and passages. Compared with Plain, Detector highlighting changed accuracy by −6.4 percentage points, with a 95% confidence interval of $[-52.1, 39.3]$. Compared with Random highlighting, the difference was −2.8 points, with a 95% confidence interval of $[-21.6, 15.9]$. Both Holm-adjusted p-values were 1.0. The confidence intervals are wide, so we find no clear evidence that Detector highlighting improved human authorship judgements.

Cannot tell responses were counted as incorrect in the accuracy analysis. Mean confidence was 3.58 for Plain, 3.30 for Detector and 3.51 for Random. Detector and Random highlighting also received similar helpfulness ratings, 3.07 and 3.08, and relevance ratings, 3.30 and 3.36.

5 Discussion

5.1 Detection Performance

The rationale BiLSTM was 0.97 percentage points below the full-context BiLSTM in F1 while using only 40% of the visible text. The confidence interval for this difference remained within the 0.02 F1 reference margin used by Ahmad et al. [1], so the loss is small relative to that benchmark. This result is also not unexpected, since Bastings et al. [2] found that their HardKuma rationale model approached full-text performance at around 40% selected text on sentiment classification.

The full-context BiLSTM improved on TF–IDF by only 0.6 F1 points, while the rationale BiLSTM was 0.4 points below it, and their AUROC scores were also very close. This means that all three models were similarly good at ranking machine-written passages above human-written ones across different decision thresholds, which suggests that word and character patterns already provide a strong signal for this dataset. TF–IDF is only a baseline for our study because it cannot select a sparse rationale from each passage or show which parts of the passage the prediction depends on.

The Transformer experiment shows that replacing the BiLSTM encoders with Transformers does not automatically improve the rationale model. The full-context Transformer performed best, while the rationale Transformer was below the rationale BiLSTM in both F1 and AUROC. Our Transformers were trained from scratch, so more training data, more model capacity, or a pretrained encoder might have improved their performance, although these were not practical within the scope of this study. Under our setup, the BiLSTM therefore gave the better result for the rationale model, and its use here also follows Bastings et al. [2], who use bidirectional recurrent encoders in the same HardKuma rationale framework.

5.2 Rationale Evidence

When the predictor used the learned rationale, it performed much better than when it was given random, first-token or last-token selections of the same size. This shows that the selector is learning to keep text that is useful for the prediction, rather than benefiting only from how much text it is allowed to keep.

Removing the selected rationale also lowered predicted-class confidence by 0.516 on average. The predictor was more confident when using the rationale alone than under the all-visible intervention. Together, these results show that the selected text carries information that the predictor relies on when making its decision.

The exact rationale was less stable across training runs than the final prediction. The three runs agreed on the predicted class for 94.1% of test passages, while the mean Jaccard similarity between their selected token positions was 0.633. Madhyastha and Jain [17] similarly found that models can make similar predictions while producing different explanations across random seeds. In our case, different runs can therefore reach the same decision while relying on somewhat different parts of the passage.

5.3 Human Evaluation

Showing the learned rationale did not improve human authorship judgements over plain text or random highlighting. Participants also rated Detector and Random highlighting almost the same for

helpfulness and relevance, so the learned highlights gave no clear advantage in how people used or perceived the evidence.

The task itself is difficult for a reader. Participants had to judge passages from different domains, including abstracts, books, news and poetry, and the machine-generated passages often closely resemble human writing. A reader may be familiar with the conventions of one domain and much less familiar with another, making it harder to judge whether every passage is authentic.

Our human result differs from GLTR, where showing token probability and rank information improved people's ability to detect generated text [9]. The two interfaces expose different kinds of information. GLTR gives readers information about how expected a token is under a language model, while our study shows the evidence selected by the detector. This difference may help explain why the benefit seen in GLTR was not observed here.

The confidence intervals are wide, so the study cannot pin down smaller positive or negative effects. The rationales clearly matter to the predictor, but we find no clear evidence that showing them helps human evaluators judge authorship.

6 Limitations

Our results describe detection within the conditions covered by RAID's four domains and six generators, all represented in training, using sampled generations. Whether the same pattern holds for unseen generators, new domains, other languages, or adversarially modified text is a question for future work.

Manual inspection during error analysis found a small number of passages labelled as English that contained non-English text. This indicates that the language metadata does not perfectly reflect the passage content and may introduce noise into the English subset.

The detection models read up to 512 tokens, which is the complete passage for 91.00% of the test set. The rationale budget of 40% applies to this visible input rather than to the small remainder of longer passages.

A removed token contributes nothing to the predictor's decision, but its position remains visible, and the selector sees the full passage before making its choice. The rationale we report is therefore the evidence the predictor used, evaluated against strong controls and a direct architectural check, rather than a claim that no other explanation of the decision is possible.

The Transformer comparison uses models of different sizes, so it tells us that the pattern we found is not specific to one architecture, without telling us whether size or design drives the difference we saw.

Twenty-five participants judging twenty passages is a small sample, and the task itself was hard. Our participants were general English speakers rather than NLP researchers or people trained to spot machine-generated text, and telling the two apart is a task even trained detectors find difficult. A larger sample, or participants with more relevant expertise, might change the result.

7 Ethical Considerations

The modelling experiments reuse RAID, an existing public research benchmark [8], released by its authors for research use. The human-written passages in RAID retain their original authorship, and we redistribute no text beyond what RAID's own release already makes available. The human study received ethics approval before recruitment. Participation was voluntary, consent was collected at the start of each form, and responses were anonymous.

A false positive labels a person's own writing as machine-generated. In a setting such as academic assessment, that carries a real cost for the author. A rationale can help someone see what the model responded to, but it does not turn a prediction into a conclusion. The model can still be wrong, and the evidence it shows can still fail to explain that error.

8 Conclusion

This study asked whether a detector can identify LLM-generated text from a sparse rationale, and whether showing that rationale can help people judge authorship. The rationale BiLSTM reached 94.5% F1 while using 40% of the visible text, compared with 95.5% for the matched full-context BiLSTM. The learned rationales also performed better than random and positional selections of the same size, and removing the selected evidence reduced the predictor's confidence. The model was therefore able to expose useful evidence while giving up only a small amount of detection performance.

Detecting machine-generated text is already difficult for automated detectors when the writing closely resembles human text, and asking people to make the same distinction is harder still. Showing participants the detector's highlights did not clearly improve their judgements over plain text or random highlighting. The model had learned evidence that helped it make the decision, but those same highlights were not enough to help a person make it.

Future work should test these findings on unseen generators, new domains and adversarially modified text. The human study should also be repeated with people who have experience in NLP, linguistics, or the type of writing being judged, to see whether domain knowledge changes how useful the rationales are. Another direction is to go beyond showing which words mattered and explain why they support a prediction. That may be what turns evidence that helps a detector into evidence that can also help the person reading it.

AI Usage Statement

I used AI assistants during this project, in line with the guidance given in CSC4019Z. I carried out the experimental design, implementation, data construction, model training, evaluation, and human study. My own code produced all results reported in this paper, and I verified them against frozen experiment artefacts.

For writing, I drafted and revised sections myself and used AI assistants (ChatGPT and Claude) for feedback, restructuring suggestions, and editing. I assessed and rewrote suggested text instead of accepting it as produced, and I rejected several suggestions that misstated a source or overstated a claim. I also used AI assistance for implementation support, including debugging and generating plotting and analysis scripts, and for checking my own reasoning about experimental provenance.

I verified all factual claims, citations, and reported numbers against the original sources and the project's experiment records.

References

[1] Uzair Ahmad, Hasan Haroglu, Algan Tezel, and Bekir Sait Ciftler. 2026. An Agentic Framework for Data Augmentation in Construction Defect Report Classification. *Advanced Engineering Informatics* 74 (2026), 104679. doi:10.1016/j.aei.2026.104679

[2] Jasmijn Bastings, Wilker Aziz, and Ivan Titov. 2019. Interpretable Neural Predictions with Differentiable Binary Variables. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, Florence, Italy, 2963–2977. doi:10.18653/v1/P19-1284

[3] A. Colin Cameron, Jonah B. Gelbach, and Douglas L. Miller. 2011. Robust Inference with Multiway Clustering. *Journal of Business & Economic Statistics* 29, 2 (2011), 238–249. doi:10.1198/jbes.2010.07136

[4] Mirko Cesarini, Lorenzo Malandri, Filippo Pallucchini, Andrea Seveso, and Frank Xing. 2024. Explainable AI for Text Classification: Lessons from a Comprehensive Evaluation of Post Hoc Methods. *Cognitive Computation* 16, 6 (2024), 3077–3095.

[5] Chun Sik Chan, Huanqi Kong, and Liang Guanqing. 2022. A Comparative Study of Faithfulness Metrics for Model Interpretability Methods. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 5029–5038. doi:10.18653/v1/2022.acl-long.345

[6] Jay DeYoung, Sarthak Jain, Nazneen Fatema Rajani, Eric Lehman, Caiming Xiong, Richard Socher, and Byron C. Wallace. 2020. ERASER: A Benchmark to Evaluate Rationalized NLP Models. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 4443–4458. doi:10.18653/v1/2020.acl-main.408

[7] Rotem Dror, Gili Baumer, Segev Shlomov, and Roi Reichart. 2018. The Hitchhiker's Guide to Testing Statistical Significance in Natural Language Processing. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Melbourne, Australia, 1383–1392. doi:10.18653/v1/P18-1128

[8] Liam Dugan, Alyssa Hwang, Filip Trhlík, Josh Magnus Ludan, Andrew Zhu, Hainiu Xu, Daphne Ippolito, and Chris Callison-Burch. 2024. RAID: A Shared Benchmark for Robust Evaluation of Machine-Generated Text Detectors. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Bangkok, Thailand, 12463–12492. https://aclanthology.org/2024.acl-long.674/

[9] Sebastian Gehrmann, Hendrik Strobelt, and Alexander M. Rush. 2019. GLTR: Statistical Detection and Visualization of Generated Text. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*. Association for Computational Linguistics, Florence, Italy, 111–116. doi:10.18653/v1/P19-3019

[10] Sture Holm. 1979. A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics* 6, 2 (1979), 65–70.

[11] Sarthak Jain, Sarah Wiegreffe, Yuval Pinter, and Byron C. Wallace. 2020. Learning to Faithfully Rationalize by Construction. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, Online, 4459–4473. doi:10.18653/v1/2020.acl-main.409

[12] John Kirchenbauer, Jonas Geiping, Yuxin Wen, Jonathan Katz, Ian Miers, and Tom Goldstein. 2023. A Watermark for Large Language Models. In *Proceedings of the 40th International Conference on Machine Learning*. PMLR, 17061–17084.

[13] Tao Lei, Regina Barzilay, and Tommi Jaakkola. 2016. Rationalizing Neural Predictions. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Austin, Texas, 107–117. doi:10.18653/v1/D16-1011

[14] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. RoBERTa: A Robustly Optimized BERT Pretraining Approach. *arXiv preprint arXiv:1907.11692* (2019). https://arxiv.org/abs/1907.11692

[15] Scott M. Lundberg and Su-In Lee. 2017. A Unified Approach to Interpreting Model Predictions. In *Advances in Neural Information Processing Systems*, Vol. 30. Curran Associates, Inc., 4765–4774. https://proceedings.neurips.cc/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html

[16] Dominik Macko, Robert Moro, Adaku Uchendu, Jason Lucas, Michiharu Yamashita, Matúš Pikuliak, Ivan Srba, Thai Le, Dongwon Lee, Jakub Simko, and Maria Bielikova. 2023. MULTITuDE: Large-Scale Multilingual Machine-Generated Text Detection Benchmark. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Singapore, 9960–9987. doi:10.18653/v1/2023.emnlp-main.616

[17] Pranava Madhyastha and Rishabh Jain. 2019. On Model Stability as a Function of Random Seed. In *Proceedings of the 23rd Conference on Computational Natural Language Learning (CoNLL)*. Association for Computational Linguistics, Hong Kong, China, 929–939. doi:10.18653/v1/K19-1087

[18] Eric Mitchell, Yoonho Lee, Alexander Khazatsky, Christopher D. Manning, and Chelsea Finn. 2023. DetectGPT: Zero-Shot Machine-Generated Text Detection using Probability Curvature. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 202)*. PMLR, 24950–24962. https://proceedings.mlr.press/v202/mitchell23a.html

[19] Bhargavi Paranjape, Mandar Joshi, John Thickstun, Hannaneh Hajishirzi, and Luke Zettlemoyer. 2020. An Information Bottleneck Approach for Controlling Conciseness in Rationale Extraction. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 1938–1952. doi:10.18653/v1/2020.emnlp-main.153

[20] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. 2014. GloVe: Global Vectors for Word Representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Doha, Qatar, 1532–1543. doi:10.3115/v1/D14-1162

[21] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language Models are Unsupervised Multitask Learners. *OpenAI Blog* 1, 8 (2019), 9.

[22] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. 2016. "Why Should I Trust You?": Explaining the Predictions of Any Classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 1135–1144. doi:10.1145/2939672.2939778

[23] Cynthia Rudin. 2019. Stop Explaining Black Box Machine Learning Models for High Stakes Decisions and Use Interpretable Models Instead. *Nature Machine Intelligence* 1, 5 (2019), 206–215. doi:10.1038/s42256-019-0048-x

[24] Sofia Serrano and Noah A. Smith. 2019. Is Attention Interpretable?. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, Florence, Italy, 2931–2951. doi:10.18653/v1/P19-1282

[25] Irene Solaiman, Miles Brundage, Jack Clark, Amanda Askell, Ariel Herbert-Voss, Jeff Wu, Alec Radford, Gretchen Krueger, Jong Wook Kim, Sarah Kreps, Miles McCain, Alex Newhouse, Jason Blazakis, Kris McGuffie, and Jasmine Wang. 2019. Release Strategies and the Social Impacts of Language Models. arXiv:1908.09203 [cs.CL] https://arxiv.org/abs/1908.09203

[26] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. In *Advances in Neural Information Processing Systems*, Vol. 30. Curran Associates, Inc., 5998–6008. https://proceedings.neurips.cc/paper/7181-attention-is-all-you-need

[27] Ronald J. Williams. 1992. Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning. *Machine Learning* 8 (1992), 229–256. doi:10.1007/BF00992696

[28] Junchao Wu, Shu Yang, Runzhe Zhan, Yulin Yuan, Lidia Sam Chao, and Derek Fai Wong. 2025. A Survey on LLM-Generated Text Detection: Necessity, Methods, and Future Directions. *Computational Linguistics* 51, 1 (2025), 275–338. doi:10.1162/coli_a_00549

[29] Junchao Wu, Runzhe Zhan, Derek F. Wong, Shu Yang, Xinyi Yang, Yulin Yuan, and Lidia S. Chao. 2024. DetectRL: Benchmarking LLM-Generated Text Detection in Real-World Scenarios. In *Advances in Neural Information Processing Systems*, Vol. 37. Curran Associates, Inc., 100369–100401. doi:10.52202/079017-3186

[30] Mo Yu, Yang Zhang, Shiyu Chang, and Tommi Jaakkola. 2021. Understanding Interlocking Dynamics of Cooperative Rationalization. In *Advances in Neural Information Processing Systems*, Vol. 34. 12822–12835.

A Supplementary Information

A.1 Experimental Configuration

The final neural models use the same surface-preserving representation. Tokens are produced with the regular expression `\w+|[^\w\s]`, the vocabulary is built from the training split only with a maximum size of 10,000 and a minimum frequency of two, and all models read the first 512 visible tokens. Token embeddings are initialised from 300-dimensional GloVe vectors and fine-tuned during training. Surface identity is preserved, and lowercasing is used only as a fallback when looking up an embedding.

The TF–IDF baseline combines word unigrams and bigrams with `char_wb` character 3–5-grams, using `min_df=2`. Its input is lowercased. Logistic regression uses balanced class weights, the `liblinear` solver, `max_iter=1000`, and random seed 42. The combined representation was selected on validation data before its reported final test evaluation.

A.1.1 HardKuma gates and rate controller. The selector outputs positive HardKuma shape parameters bounded by $a \in [0.01, 2.0]$ and $b \in [0.1, 10.0]$. During training, a Kumaraswamy sample is reparameterised from $u \sim \mathcal{U}(0, 1)$, stretched from $(0, 1)$ to $(-0.1, 1.1)$,

Table 6: Final neural-model configurations. Parameter counts are trainable parameters in the reported systems.

	Full BiLSTM	Rationale BiLSTM	Full Transformer	Rationale Transformer
Visible context	512	512	512	512
Embedding dim.	300	300	300	300
Encoder	BiLSTM	selector + predictor BiLSTMs	2-layer Transformer	selector + predictor Transformers
Hidden / model dim.	128 per direction	128 per direction	300	300
Attention heads	–	–	4	4
Feed-forward dim.	–	–	600	600
Dropout	0.2	0.2	0.2	0.2
Batch size	32	32	32	32
Maximum epochs	25	25	25	25
Optimiser	Adam	Adam	Adam	Adam
Learning rate	4×10^{-4}	4×10^{-4}	4×10^{-4}	4×10^{-4}
Gradient clip	5	5	5	5
Warm-up	none	controller only	10% LR warm-up	10% LR warm-up
Seeds	42, 43, 44	42, 43, 44	42, 43, 44	42, 43, 44
Parameters	3,440,834	3,881,668	4,600,802	6,201,604

and clipped to $[0, 1]$, following the HardKuma construction of Bastings et al. [2]. The controller targets the expected non-zero gate rate. With target rate r^* and violation $v = r(z) - r^*$, the optimised loss is

$$\mathcal{L} = \mathcal{L}_{\text{cls}} + \lambda v + \frac{\mu}{2} v^2.$$

The final BiLSTM rationale configuration uses $\mu = 10$, a 300-step controller warm-up, an exponential moving average with $\alpha = 0.5$, $\lambda_0 = 0.02$, and $\lambda \in [10^{-4}, 1]$. The multiplier is updated in log space with step size 0.02 from the smoothed rate violation. The final model uses no continuity or coherence penalty.

At validation and test time, selection is deterministic. The median gate score at $u = 0.5$ ranks non-padding positions, and stable top-k selection keeps $k = \lceil r^* n \rceil$ positions for a passage with n visible tokens. Padding is never eligible, and exact score ties are resolved by stable order. The final test set required no empty-selection fallback.

Algorithm 1 makes the two modes explicit. During training the gates are stochastic, so gradients reach the selector through the reparameterised sample. At evaluation the sampling is replaced by a fixed budget, so a passage of n visible tokens always yields $\lceil r^* n \rceil$ selected positions. In both modes an excluded position stays in the sequence carrying a zero vector, which leaves sequence length and gap structure visible to the predictor while its token content is not.

A.1.2 Checkpoint eligibility. Checkpoint selection uses validation machine-class F1 subject to a pre-specified rate constraint. A rationale checkpoint is eligible only when its deterministic validation selected fraction is at most $1.25\, r^*$. Among eligible epochs, the earliest epoch attaining the highest validation F1 is retained. All final reported rationale checkpoints satisfied this rule.

Algorithm 1 Selector–predictor forward pass. Training samples stochastic gates; evaluation applies a deterministic budget.

Require: token ids $x_{1:n}$, target rate r^*, mode $\in$ {train, eval}
Ensure: class posterior $p(y \mid \tilde{e}_{1:n})$

$e_{1:n} \leftarrow \textsc{Embed}(x_{1:n})$
$h_{1:n} \leftarrow \textsc{SelectorBiLSTM}(e_{1:n})$
for $i = 1$ **to** n **do**
  $a_i \leftarrow 0.01 + 1.99\,\sigma(W_a h_i)$ ▷ bound shape to $[0.01, 2.0]$
  $b_i \leftarrow 0.10 + 9.90\,\sigma(W_b h_i)$ ▷ bound shape to $[0.1, 10.0]$
end for
if mode = train **then**
  for $i = 1$ **to** n **do**
    $u_i \sim \mathcal{U}(0, 1)$
    $k_i \leftarrow \left(1 - (1 - u_i)^{1/b_i}\right)^{1/a_i}$ ▷ Kumaraswamy sample
    $s_i \leftarrow -0.1 + 1.2\, k_i$ ▷ stretch to $(-0.1, 1.1)$
    $z_i \leftarrow \min(1, \max(0, s_i))$ ▷ clip; mass at exactly 0 and 1
  end for
else
  $s_{1:n} \leftarrow \textsc{MedianGate}(a_{1:n}, b_{1:n})$ ▷ deterministic score at $u = 0.5$
  $k \leftarrow \lceil r^* n \rceil$
  $S \leftarrow$ indices of the k highest s_i over non-padding positions
  $z_i \leftarrow \mathbb{1}[i \in S]$ for all i
end if
$\tilde{e}_i \leftarrow z_i e_i$ for all i ▷ excluded positions zeroed, not removed
$g \leftarrow \textsc{PredictorBiLSTM}(\tilde{e}_{1:n})$
return $\textsc{Classify}(\textsc{MaskedMeanPool}(g))$

A.2 Validation and Additional Results

A.2.1 Faithfulness intervention definitions. Sufficiency and comprehensiveness are computed with the trained rationale predictor itself. Let c be the class predicted from the learned rationale. Three forward passes preserve the original visible positions: the learned rationale, an all-visible intervention, and the complement of the learned rationale. The reported diagnostics are

$$\text{Sufficiency} = p(c \mid \text{all}) - p(c \mid \text{rationale}),$$

$$\text{Comprehensiveness} = p(c \mid \text{all}) - p(c \mid \text{complement}).$$

Table 7: Validation-only context-length comparison for the full-context BiLSTM. Coverage is the percentage of validation passages that fit completely within the visible context.

Tokens	F1 mean ± SD	Coverage	Runtime (s)
256	0.9397 ± 0.0017	42.83%	2,786.69
512	0.9571 ± 0.0051	91.00%	4,862.36
1,024	0.9578 ± 0.0030	98.17%	10,450.91

Table 8: Validation-only rationale-budget comparison at 512 visible tokens. Mean tokens are measured on the validation split, so they differ slightly from the test-split figures reported in Section 4. Forty percent was the strongest of the three tested sparse settings; no global optimum is claimed.

Target	F1 mean ± SD	Achieved	Mean tokens
20%	0.9096 ± 0.0178	20.17%	60.32
30%	0.9274 ± 0.0123	30.18%	90.27
40%	0.9409 ± 0.0024	40.16%	120.16

Table 9: Machine-class F1 by domain on the final test set. Neural values are mean ± sample SD across seeds 42, 43, and 44. Every domain occurs in training.

Domain	TF–IDF	Full BiLSTM	Rationale BiLSTM
Abstracts	0.9365	0.9454 ± 0.0020	0.9355 ± 0.0051
Books	0.9701	0.9659 ± 0.0102	0.9570 ± 0.0056
News	0.9408	0.9550 ± 0.0055	0.9539 ± 0.0086
Poetry	0.9467	0.9533 ± 0.0076	0.9350 ± 0.0058

Table 10: Machine recall by generator on the final test set. Generator subsets contain only machine-generated passages, so these values are recall rather than binary F1. Every generator occurs in training.

Generator label	TF–IDF	Full BiLSTM	Rationale BiLSTM
`chatgpt`	0.9100	0.9500 ± 0.0000	0.9433 ± 0.0115
`gpt2`	0.9100	0.9433 ± 0.0153	0.9467 ± 0.0231
`gpt4`	0.9300	0.9233 ± 0.0058	0.9200 ± 0.0300
`llama-chat`	0.9900	0.9900 ± 0.0100	0.9767 ± 0.0153
`mistral`	0.9200	0.9567 ± 0.0058	0.9267 ± 0.0252
`mistral-chat`	0.9700	0.9867 ± 0.0058	0.9867 ± 0.0058

Sufficiency near zero means the rationale preserves the confidence of the all-visible intervention, and a negative value means the learned rationale produces higher predicted-class confidence. Higher comprehensiveness means removing the selected evidence reduces confidence more. These are intervention diagnostics rather than proof of a complete causal explanation, because the predictor trains on selected-input patterns while the all-visible and complement inputs fall outside that distribution.

A.3 Reliability Checks

A.4 Human Study Materials

Detector highlighting uses the exact surface-token positions selected by the final seed-44 rationale model. Random highlighting uses the same number of positions for the same passage, so the two highlighted conditions differ in where evidence is placed rather than how much text is marked. The 20-passage pool was deliberately stratified by model outcome for diagnostic coverage and is not a prevalence sample of detector errors.

Table 11: Reliability checks supporting the final reported pipeline.

Check	What was verified
Split leakage	No source IDs, row IDs, or exact texts are shared across train, validation, and test splits.
Surface recovery	Stored token offsets recover the original source text used for rationale highlighting.
BiLSTM sequence handling	Packed-sequence, padding, and batch-composition checks confirm that padding or batch membership does not change final BiLSTM predictions.
Top-k selection	Deterministic selection returns the requested per-example count and never selects padding.
Excluded-token intervention	With the selected mask and valid sequence length fixed, changing an excluded token's identity does not change the predictor output. This checks the claim that unselected token content has no direct path to the predictor.
Selection fallback	None of the 1,200 final test passages required the empty-selection fallback.
Configuration freeze	Final reported configurations were chosen using training and validation evidence and frozen before their reported test evaluation. Earlier superseded development had examined the same v3 test partition, so the partition is not described as untouched throughout the project.

Table 12: Human-study design and response structure.

Item	Final design
Passages	20 unique test passages; 10 Human and 10 Machine
Outcome strata	5 correct-Human, 5 correct-Machine, 5 false-positive, 5 false-negative
Forms	6 forms, 10 distinct passages per form
Conditions	Plain, Detector, matched-count Random
Counterbalancing	Each passage appears once in each condition across three different forms
Judgement	Human, Machine, or Cannot tell
Ratings	Confidence 1–5; highlighted conditions also rate helpfulness and relevance 1–5
Quality control	Practice item and attention check on each form
Analysed sample	25 participants, 250 retained judgements

A.5 Example Rationale Output

The four passages below show examples from the final rationale model. Yellow highlighting marks the tokens selected by the model and passed to the predictor. The false positive and false negative are the two errors shown as excerpts in Section 4. Correct human and correct machine examples are included for comparison.

The examples show that the selected evidence is spread across the passages and includes both technical terminology and more general academic language. The two errors also show that similar kinds of text can support different predictions, which is consistent with the cases discussed in the main Error Analysis.

Directional infrared emission at 1367 and 5230 nm is generated in Rb vapours that are step-wise excited by low-power resonant light. The mid-infrared radiation originating from amplified spontaneous emission on the 5D-6P transition consists of forward- and backward-directed components with distinctive spectral and spatial properties. Diffraction limited near-infrared light at 1367 nm generated in the co-propagating direction only is a product of parametric wave mixing around the 5P-5D-6P-6S-5P transition loop. This highly non-degenerate mixing process involves one externally applied and two internally generated optical fields. Similarities between wave mixing generated blue and near-IR light are demonstrated.

Figure 3: Correct human example. The passage was assigned $p(M) = 0.0000012$. The passage contains 128 visible tokens, of which 52 were selected.

The title refers to an improved Directivity Analysis method which is a variant of Mutual Coupling Based Linear Array (MCBLA). This technique enhances performance estimation in direction-of-arrival (DoA) measurement that utilizes a digital steerable antenna. It's particularly effective at suppressing sidelobe noise, improving tracking accuracy, and detecting weak signals compared to other conventional approaches. A series of simulations was carried out on the MCBLA with two signal sources over an infinite plane with free space propagation in order to study its impact on clutter rejection and DoA estimates. Overall findings conclude that the proposed technique has enhanced accuracy, robustness and reliability as compared to preeminent methods and may have potential applications for automotive and military systems.

Figure 4: Correct machine example. The passage was assigned $p(M) = 1.000$. The passage contains 134 visible tokens, of which 54 were selected.

Solving crystal structures from kinematical X-ray or electron diffraction patterns of single crystals requires many more diffracted beams to be recorded than there are atoms in the structure, since the phases of the structure factors can only be retrieved from such data if the atoms can be resolved as sharply peaked objects. Here a method is presented by which the fact that multiple scattering encodes structure factor phases in the diffracted intensities is being used for solving the crystallographic phase problem. The retrieval of both amplitudes and phases of electron structure factors from diffraction patterns recorded with varying angle of incidence will be demonstrated. No assumption about the scattering potential itself is being made. In particular, the resolution in the diffraction data does not need to be sufficient to resolve atoms, making this method potentially interesting for electron crystallography of 2-dimensional protein crystals and other beam-sensitive complex structures.

Figure 5: False positive. The human-written passage was assigned $p(M) = 0.720$. The passage contains 162 visible tokens, of which 65 were selected.

In this paper, we study tangentially biharmonic Lagrangian H-umbilical submanifolds in complex space forms. We define the tangentially biharmonic Lagrangian H-umbilical submanifolds as those whose mean curvature vector field is parallel to the tangent bundle. We provide a necessary and sufficient condition for a submanifold to be tangentially biharmonic Lagrangian H- umbilical. Moreover, we obtain various characterizations and properties of these submanifolds. We also investigate the classification of these submanifolds in complex space forms. Finally, we give examples of tangentially biharmonic Lagrangian H-umbilical submanifolds and discuss their geometric properties.

Figure 6: False negative. The machine-generated passage was assigned $p(M) = 0.0004$. The passage contains 105 visible tokens, of which 42 were selected.